

# Datenbanken II

Andreas Nicola

Mail: [andreas.nicola@sap.com](mailto:andreas.nicola@sap.com)

# Agenda

- SQL Injection
- Index
- Partitionierung
- Materialisierte Sichten

# SQL – Structured Query Language

# SQL – Structured Query Language



Nicht jedes Datenbanksystem unterstützt alle SQL-Befehle, die Folien fokussieren sich primär auf MySQL, andere Datenbanksysteme wie z.B. Oracle, MariaDB, PostgreSQL, etc. können sich leicht von der aufgeführten Syntax unterscheiden. Ggf. sind manche Befehle in verschiedenen Systemen gar nicht implementiert und daher nicht möglich.

# SQL – Namenskonvention

- GROSSBUCHSTABEN → Reservierte Bezeichner / Befehle
- <x> → Platzhalter für Bezeichner (Buchstaben, Ziffern, Unterstrich) oder Literal
- [ x ] → optional
- [ x | y | z ] → Auswahlliste (entweder oder), optional
- { x | y | z } → Auswahlliste (entweder oder), obligatorisch
- ... → Wiederholung

# SQL – SQL-Injection

# SQL – SQL-Injection

SQL Injection bezeichnet den Versuch bzw. die tatsächliche Durchführung durch gezielte Eingaben SQL Queries zu erzeugen welche genutzt werden um Angriffe auf die Infrastruktur auszuführen.

So ist es z.B. möglich durch SQL Injections Datenbanktabellen zu löschen.

Dies muss daher durch Prüfungen der Eingabe unbedingt verhindert werden.

Eine entsprechende Eingabe mit Anwendung im SQL Statement wäre:

Eingabe Matrikelnummer

1234

## **Ergebnis:**

```
SELECT Vorname, Nachname FROM Studierende WHERE Matrikelnummer = 1234;
```

# SQL – SQL-Injection

Angenommen statt der Matrikelnummer „1234“ wird zusätzlich Text in das Eingabefeld eingetragen.

Eingabe Matrikelnummer

**1234 OR 1 = 1**

## Ergebnis:

SELECT Vorname, Nachname FROM Studierende WHERE Matrikelnummer = **1234 OR 1 = 1**

**1 = 1** → immer WAHR! Durch Veroderung gesamtes WHERE immer WAHR

Dies hat zur Folge, dass alle Studierenden als Ergebnis angezeigt werden und Hacker so an alle Daten kommen können. Im schlimmsten Fall sind E-Mail Adressen oder Passwörter Teil der Daten

---

Gleiches gilt für mehrere Attribute.

Eingabe Stadt

**, ' OR „“ = ,'**

Eingabe Land

**, ' OR „“ = ,'**

## Ergebnis:

SELECT Vorname, Nachname FROM Studierende WHERE Stadt = **, ' OR „“ = ,'** AND Land = **, ' OR „“ = ,'**

**„“ = ,'** → immer WAHR! Durch Veroderung gesamtes WHERE immer WAHR

# SQL – SQL-Injection

Eine der größten Gefahren ist das Löschen einer Datenbanktabelle, damit kann großer Schaden in den entsprechenden Systemen angerichtet werden, z.B. werden alle Kundendaten gelöscht, kann sich niemand mehr anmelden.

Eine entsprechende Eingabe mit Anwendung im SQL Statement wäre:

Eingabe Matrikelnummer

**1234; DROP TABLE Kunden;**

## **Ergebnis:**

SELECT Vorname, Nachname FROM Studierende WHERE Matrikelnummer = **1234;**  
**DROP TABLE Kunden;**

Natürlich kennen Hacker die Namen der Datenbanktabellen nicht, da diese allerdings i.d.R. gewissen Regeln folgen und oftmals identisch sind kann man diese gut erraten. Ebenso ist es möglich einfach möglichst viele Namen zu testen in der Hoffnung einige Namen tatsächlich zu treffen.

# SQL – SQL-Injection

## Mögliche Lösungen um potentielle SQL Injections zu vermeiden sind:

- Bestimmte Keywords in der Eingabe verbieten  
(Eingabe auf DROP prüfen und Anfrage ablehnen sollte es Teil des Textes sein)
- Eingabe durch Anführungszeichen umklammern, damit wird die gesamte Eingabe zum WHERE  
→ ,1234; DROP TABLE Kunden;  
→ SQL:  
SELECT Vorname, Nachname FROM Studierende WHERE Matrikelnummer = ,1234; DROP TABLE Kunden;‘
- Arbeiten mit Parametern, dabei werden Teile der SQL Query durch Parameter ersetzt und diese durch Eingaben, ähnlich zu umklammern mit Anführungszeichen
- Prüfungen, ob die Eingabe zu entsprechender Stelle passt, bei Auswahl Spalten beispielsweise nur einzelne Wörter (Spaltennamen) erlauben

# SQL – Parameter Binding

SELECT Vorname, Nachname FROM Studierende WHERE Stadt = ? AND Geburtsdatum < ?

| Matrikelnummer | Nachname   | Vorname | Geb. Datum | Stadt        |
|----------------|------------|---------|------------|--------------|
| 1              | Meier      | Moritz  | 01.01.2000 | Mannheim     |
| 2              | Müller     | Max     | 01.01.1990 | Heidelberg   |
| 3              | Mustermann | Mark    | 01.01.1950 | Ludwigshafen |

## Pseudo-Code für das Binding der Parameter:

```
query = "SELECT Vorname, Nachname FROM Studierende WHERE Stadt = ? AND Geburtsdatum < ?";  
statement = connection->prepare(query);  
statement->bind_parameter( 1, "Mannheim" );  
statement->bind_parameter( 2, "2005-01-01" );  
statement->execute( );
```

## Ergebnis:

| Matrikelnummer | Nachname | Vorname | Geb. Datum | Stadt    |
|----------------|----------|---------|------------|----------|
| 1              | Meier    | Moritz  | 01.01.2000 | Mannheim |

# SQL – SQL-Injection



SQL Injection Done Right.

Quelle: [SQL Injection Heizaman](#)

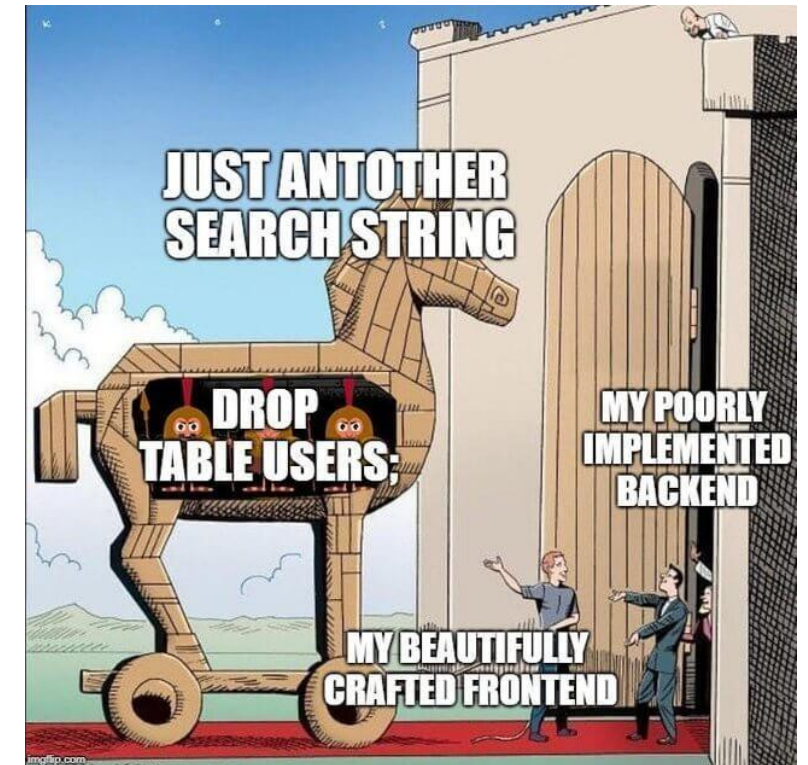
## SQL INJECTION



Quelle: [SQL Injection Heizaman](#)



Quelle: <https://xkcd.com/327/>



Quelle: [SQL Injection Heizaman](#)

# SQL – Index

# SQL – Index

Ein Index in einer Datenbank ist eine Struktur zur Beschleunigung von Suchvorgängen, in der Daten auf- oder absteigend sortiert angelegt werden. Primärschlüssel und UNIQUE Spalten erzeugen automatisch einen Index.

Erzeugung eines Index erfordert Zeit und benötigt Speicherplatz

- → Schnellerer Lesezugriff
- → Langsamerer Schreibzugriff (Aktualisierung/Aufbau des Index)

→ JOIN-Spalten sollten mit Indizes versehen sein

---

```
CREATE [ UNIQUE | FULLTEXT | SPATIAL ] INDEX  
<Index_Name>  
ON <Datenbanktabelle>  
(<Spaltenname> [ ASC | DESC ], [...]);
```

| Befehl   | Erklärung                                               |
|----------|---------------------------------------------------------|
| UNIQUE   | Eindeutigkeit                                           |
| FULLTEXT | Volltextsuche in Textspalten<br>(z. B. MATCH...AGAINST) |
| SPATIAL  | Geodaten (mit GEOMETRY, POINT, etc.)                    |

# SQL – Index

## Anzeige der Indizes einer Datenbanktabelle:

**SHOW INDEX FROM** <Datenbanktabelle>;

```
SELECT *  
FROM information_schema.STATISTICS  
WHERE TABLE_SCHEMA = <Datenbank> AND TABLE_NAME = <Datenbanktabelle>;
```

## Löschen eines Index:

**DROP INDEX** <Index\_Name>  
**ON** <Datenbanktabelle>;

## Ausführungsplan und Index-Nutzung:

**EXPLAIN** SELECT \* FROM Besucher  
WHERE email = 'maria@muster.com';

# SQL – Index – Anwendungsbeispiel

## Fall 1

Da Mitarbeiter des Freizeitparks auch häufig über den Nachnamen (ggf. auch in Kombination mit dem Geburtsdatum) gesucht werden, soll für beide Fälle ein Index definiert werden, um schnellere Zugriffe auf die Mitarbeiter-Datenbanktabelle zu ermöglichen.

```
CREATE INDEX Index_Nachname  
ON Mitarbeiter(Nachname);
```

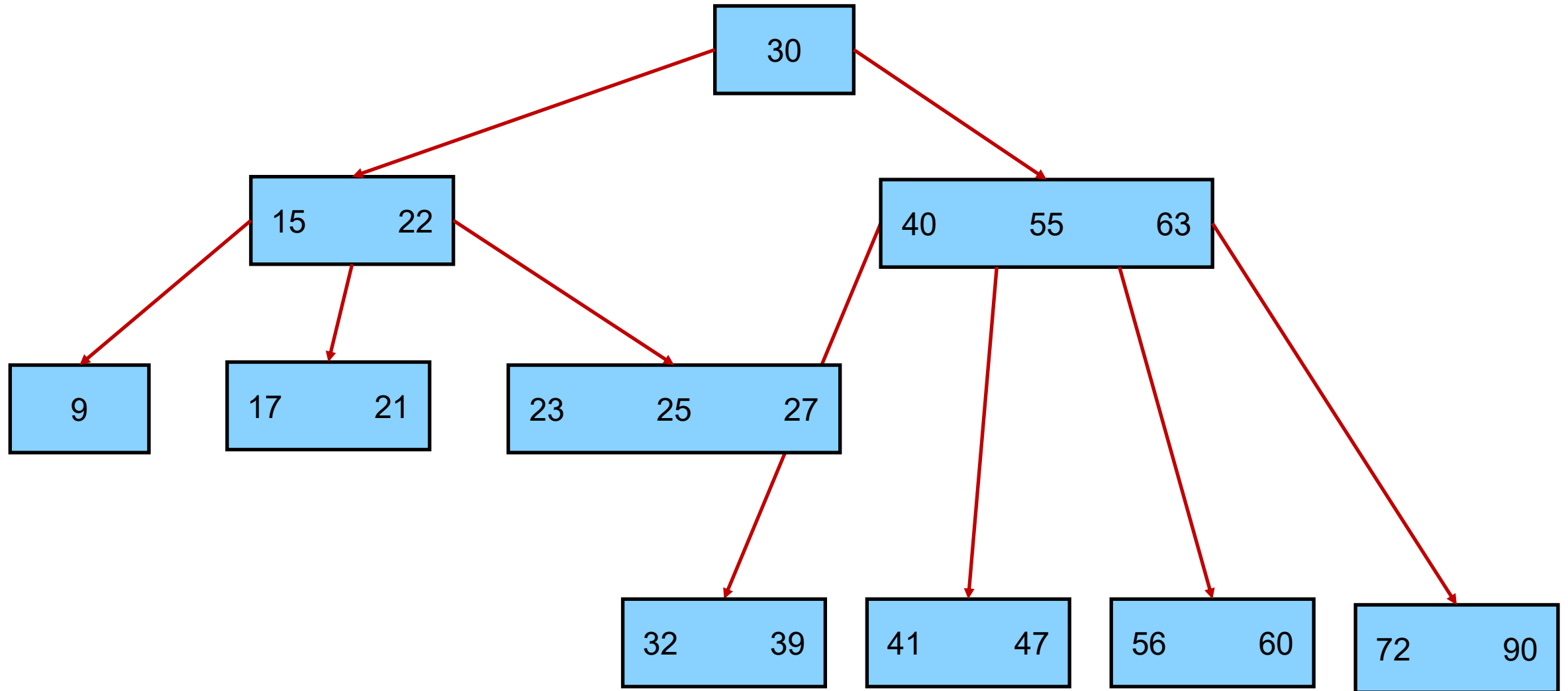
```
CREATE INDEX Index_Nachname_Geburtsdatum  
ON Mitarbeiter(Nachname, Geburtsdatum);
```

## Fall 2

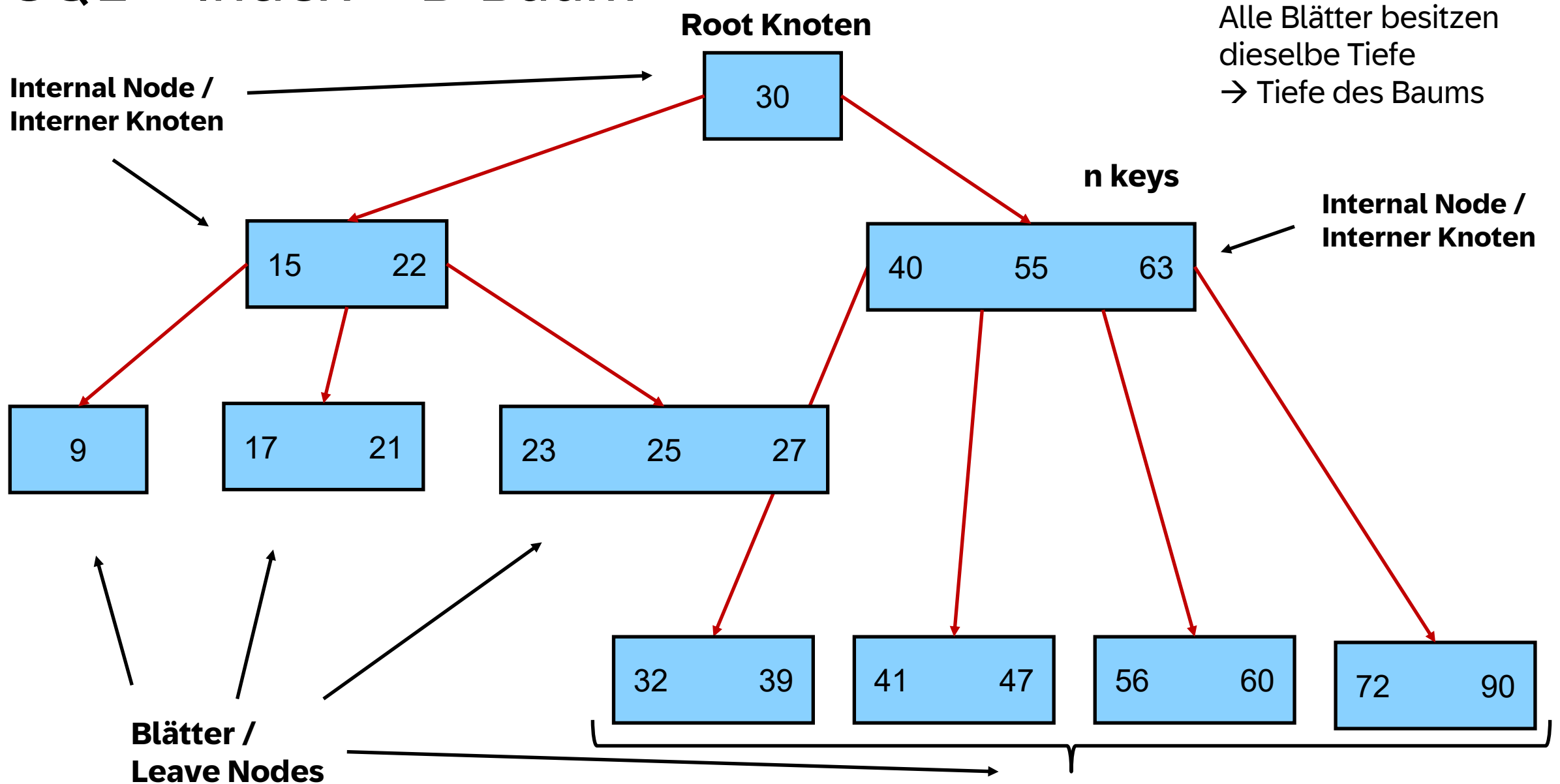
Die Besucher müssen sich nun mit einer E-Mail Adresse registrieren, da diese genutzt wird um sich anzumelden und daher auch häufig für die Selektion, soll ein eindeutiger Index definiert werden.

```
CREATE UNIQUE INDEX Index_EMail  
ON Besucher(EMail);
```

# SQL – Index – B-Baum

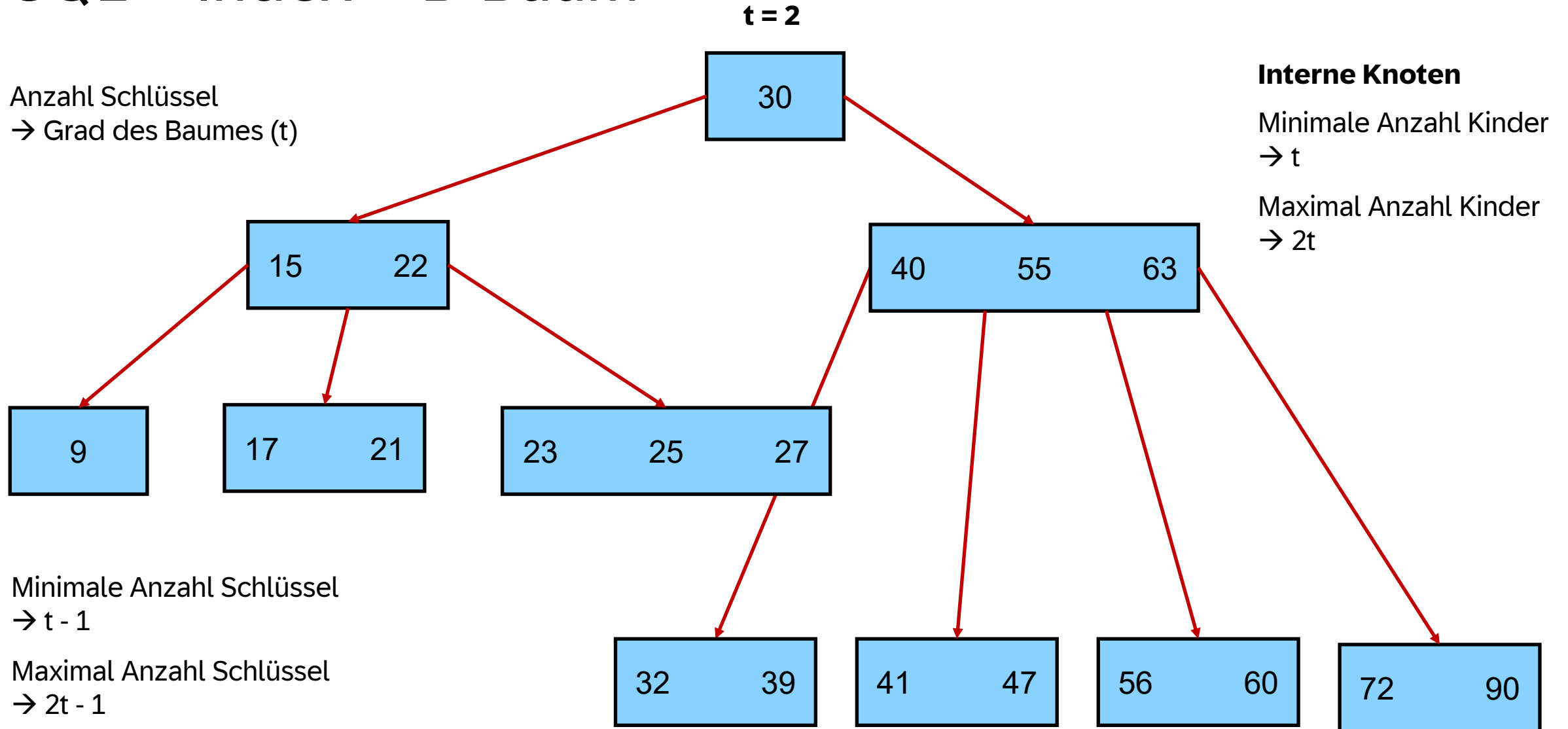


# SQL – Index – B-Baum



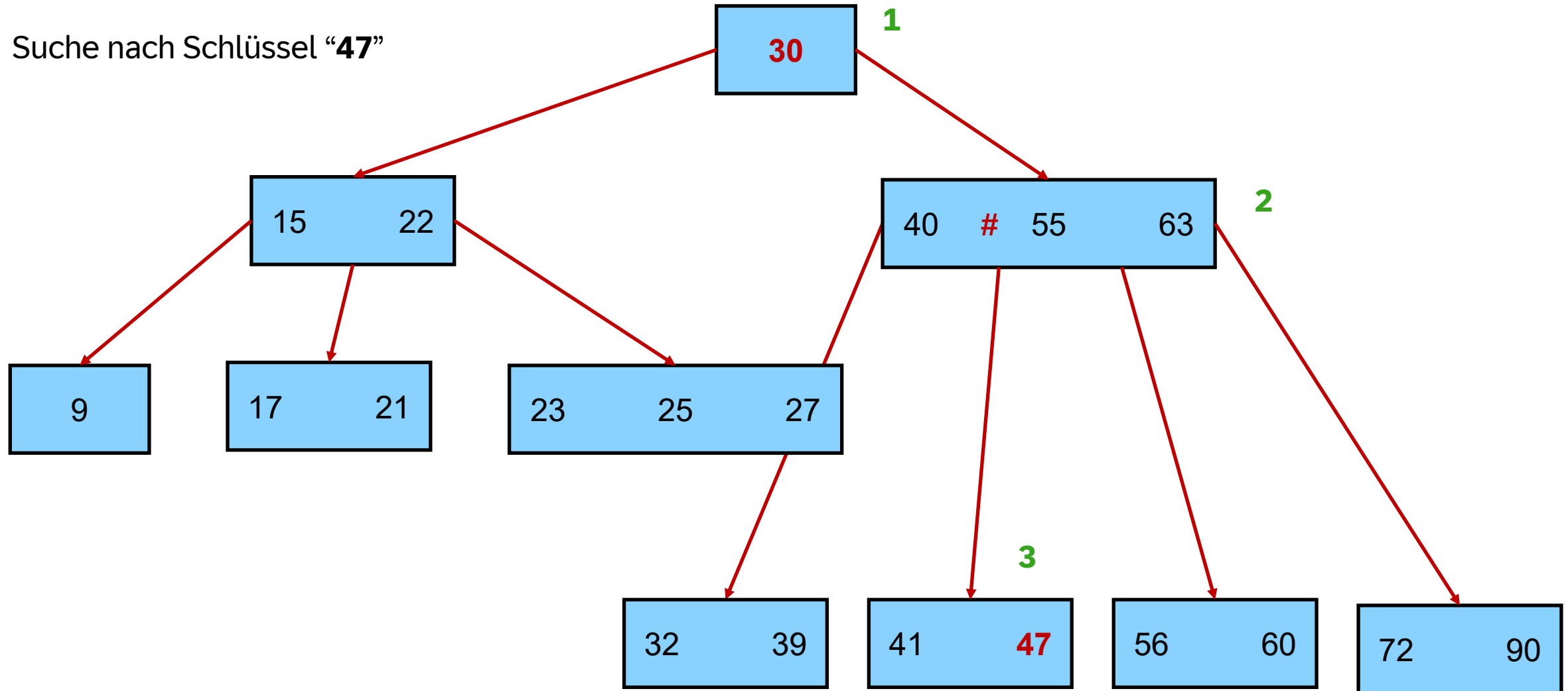
# SQL – Index – B-Baum

Anzahl Schlüssel  
→ Grad des Baumes ( $t$ )



# SQL – Index – B-Baum

Suche nach Schlüssel "47"



# Aufgabe 01

# Aufgabe 01

1. Erzeugen Sie einen Index für die Kombination aus Vorname und Nachname der Besucher/-innen.
2. Erzeugen Sie einen Index für das Geburtsdatum der Mitarbeitenden.

# SQL – Partitionierung

# SQL – Partitionierung

Partitionierung ist eine horizontale, vollständige und transparente Aufteilung einer Relation in disjunkte Teilrelationen. Diese Teilrelationen werden als Partitionen bezeichnet.

## Große Relationen

- Keine Parallelzugriffe, da nur eine Relation und somit ein Speicher
- Suche ohne Index durchsucht komplette Relation obwohl ggf. nicht notwendig

## Vorteile Partitionierung

- Transparenz (kein Unterschied für Anwendung und SQL)
- Parallele Verarbeitung
- Gezielter Zugriff auf nur eine oder wenige Partitionen statt auf gesamte Relation

→ (teilweise erhebliche) Performanceverbesserung

| Begriff                      | Bedeutung                                                                                          |
|------------------------------|----------------------------------------------------------------------------------------------------|
| Horizontale Partitionierung  | Aufteilung der Zeilen in verschiedene Partitionen                                                  |
| Vollständige Partitionierung | Alle Daten sind partitioniert, es gibt keine unpartitionierten Bereiche                            |
| Transparente Partitionierung | Anwendung und SQL sehen nur die logische Tabelle, Partitionen werden automatisch behandelt         |
| Disjunkte Partitionierung    | Keine redundante Aufteilung der Daten, jeder Eintrag existiert nur einmal in einer der Partitionen |

# SQL – Partitionierung

| Partitionierung           | Erklärung                                                                                                                                                                              | MySQL |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Bereichspartitionierung   | <ul style="list-style-type: none"><li>- Einteilung in disjunkte Bereiche (oft zeitabhängig, z.B. nach Jahren)</li></ul>                                                                | ✓     |
| Intervallpartitionierung  | <ul style="list-style-type: none"><li>- spezielle Bereichspartitionierung</li><li>- Vorgabe der Intervalle und automatisches Erzeugen neuer Partition (z.B. Monatsintervall)</li></ul> |       |
| List-Partitionierung      | <ul style="list-style-type: none"><li>- Einteilung nach Listen (z.B. Vorlesungen oder Kurse)</li></ul>                                                                                 | ✓     |
| Hash-Partitionierung      | <ul style="list-style-type: none"><li>- Einteilung nach Hash-Codes</li><li>- nur, falls keine andere sinnvolle Einteilung möglich</li></ul>                                            | ✓     |
| Schlüssel-Partitionierung | <ul style="list-style-type: none"><li>- Spezielle Hash-Partitionierung</li><li>- Primär-/Sekundärschlüssel als Hash-Code</li></ul>                                                     | ✓     |

Partitionierungen sind auch in Kombinationen möglich, damit entstehen Unter-Partitionen (Sub-Partition) in der jeweiligen Partition.

- Hash → Range
  - zuerst Hash-Partitionierung, dann Bereichspartitionierung
- Hash → List
  - zuerst Hash-Partitionierung, dann List-Partitionierung
- Range → List
  - zuerst Bereichspartitionierung, dann List-Partitionierung

# SQL – Referenzpartitionierung

| Matrikelnummer | Nachname | Vorname |
|----------------|----------|---------|
| 1              | Meier    | Moritz  |
| 2              | Müller   | Max     |

Datenbanktabelle Studierende ist partitioniert über die Matrikelnummer.

| Matrikelnummer | Nachname | Vorname |
|----------------|----------|---------|
| 1              | Meier    | Moritz  |

| Matrikelnummer | Nachname | Vorname |
|----------------|----------|---------|
| 2              | Müller   | Max     |

Partitionierung einer durch Fremdschlüssel abhängigen Datenbanktabelle durch die jeweiligen Primärschlüssel.

→ Note soll auch partitioniert werden, in Abhängigkeit zu Matrikelnummer

```
CREATE TABLE Noten (  
  Matrikelnummer INT PRIMARY KEY  
  Vorlesung VARCHAR(25) NOT NULL,  
  Note NUMERIC(8,2),  
  CONSTRAINT Student_FK FOREIGN KEY(Matrikelnummer) REFERENCES Matrikelnummer  
)  
PARTITION BY REFERENCE(Student_FK);
```

| Matrikelnummer | Vorlesung            | Note |
|----------------|----------------------|------|
| 1              | Datenbanken          | 1,8  |
| 1              | Software Engineering | 3,1  |
| 2              | Datenbanken          | 2,5  |
| 2              | Software Engineering | 1,3  |

Partitionierung passend zu den Datensätzen der Primärschlüsseltabelle  
→ Referenzpartitionierung

| Matrikelnummer | Vorlesung            | Note |
|----------------|----------------------|------|
| 1              | Datenbanken          | 1,8  |
| 1              | Software Engineering | 3,1  |

p0

| Matrikelnummer | Vorlesung            | Note |
|----------------|----------------------|------|
| 2              | Datenbanken          | 2,5  |
| 2              | Software Engineering | 1,3  |

p1

# SQL – Partitionierung – SQL

```
CREATE TABLE Mitarbeiter (  
  Vorname VARCHAR(25) NOT NULL,  
  Nachname VARCHAR(25) NOT NULL,  
  Benutzername VARCHAR(16) NOT NULL,  
  EMail VARCHAR(35),  
  Eintrittsdatum DATE NOT NULL  
)  
PARTITION BY RANGE( YEAR(Eintrittsdatum))  
(  
  PARTITION p0 VALUES LESS THAN (1960),  
  PARTITION p1 VALUES LESS THAN (1970),  
  PARTITION p2 VALUES LESS THAN (1980),  
  PARTITION p3 VALUES LESS THAN (1990),  
  PARTITION p4 VALUES LESS THAN MAXVALUE  
);
```



MySQL erlaubt nur  
Partitionen basierend auf  
PK/Unique Index wenn  
diese vorhanden sind.

```
CREATE TABLE Mitarbeiter (  
  Personalnummer INT NOT NULL,  
  Vorname VARCHAR(30),  
  Nachname VARCHAR(30),  
  Eintrittsdatum DATE NOT NULL DEFAULT '1970-01-01',  
  Standort_Nummer INT  
)  
PARTITION BY LIST(Standort_Nummer)  
(  
  PARTITION pNorth VALUES IN (3,5,6,9,17),  
  PARTITION pEast VALUES IN (1,2,10,11,19,20),  
  PARTITION pWest VALUES IN (4,12,13,14,18),  
  PARTITION pCentral VALUES IN (7,8,15,16)  
);
```

```
CREATE TABLE Mitarbeiter (  
  Vorname VARCHAR(25) NOT NULL,  
  Nachname VARCHAR(25) NOT NULL,  
  Benutzername VARCHAR(16) NOT NULL,  
  EMail VARCHAR(35),  
  Eintrittsdatum DATE NOT NULL  
)  
PARTITION BY KEY(Eintrittsdatum) PARTITIONS 6;
```

# SQL – Partitionierung – Anwendungsbeispiel

| Matrikelnummer | Nachname   | Vorname | Geb. Datum |
|----------------|------------|---------|------------|
| 1              | Meier      | Moritz  | 01.01.2000 |
| 2              | Müller     | Max     | 01.01.1990 |
| 3              | Mustermann | Mark    | 01.01.2003 |
| 4              | Bauer      | Julia   | 01.01.1998 |

```
CREATE TABLE Student (  
  Matrikelnummer INT NOT NULL,  
  Nachname VARCHAR(25) NOT NULL,  
  Vorname VARCHAR(16) NOT NULL,  
  Geburtsdatum DATE NOT NULL  
)  
PARTITION BY RANGE( YEAR(Geburtsdatum) )  
(  
  PARTITION p0 VALUES LESS THAN (2000),  
  PARTITION p1 VALUES LESS THAN MAXVALUE  
);
```

**p1**

**p0**

## Partition 01

| Matrikelnummer | Nachname | Vorname | Geb. Datum |
|----------------|----------|---------|------------|
| 2              | Müller   | Max     | 01.01.1990 |
| 4              | Bauer    | Julia   | 01.01.1998 |

## Partition 02

| Matrikelnummer | Nachname   | Vorname | Geb. Datum |
|----------------|------------|---------|------------|
| 1              | Meier      | Moritz  | 01.01.2000 |
| 3              | Mustermann | Mark    | 01.01.2003 |

Auf Grund der großen Anzahl Studierender soll die Datenbanktabelle partitioniert werden. Das Geburtsdatum eignet sich auf Grund der Zeitabhängigkeit und Selektivität gut dafür.

# SQL – Partitionierung – Anzeige

## SELECT

TABLE\_SCHEMA,  
TABLE\_NAME,  
PARTITION\_NAME,  
SUBPARTITION\_NAME,  
PARTITION\_ORDINAL\_POSITION,  
SUBPARTITION\_ORDINAL\_POSITION,  
PARTITION\_METHOD,  
SUBPARTITION\_METHOD,  
PARTITION\_EXPRESSION,  
SUBPARTITION\_EXPRESSION,  
TABLE\_ROWS

## FROM

information\_schema.PARTITIONS

## WHERE

TABLE\_SCHEMA = <Schema>  
AND TABLE\_NAME = <Datenbanktabelle>;

| TABLE_SCHEMA | TABLE_NAME  | PARTITION_NAME | PARTITION_METHOD | PARTITION_EXPRESSION | TABLE_ROWS |
|--------------|-------------|----------------|------------------|----------------------|------------|
| <Datenbank>  | Mitarbeiter | p0             | RANGE            | Eintrittsdatum       | 10000      |
| <Datenbank>  | Mitarbeiter | p1             | RANGE            | Eintrittsdatum       | 15000      |
| <Datenbank>  | Mitarbeiter | p2             | RANGE            | Eintrittsdatum       | 12000      |

# SQL – Partitionierung – Änderung

Partitionierungen können auch nachträglich modifiziert werden, allerdings nur innerhalb der jeweiligen Art, es ist daher nicht möglich eine RANGE-Partitionierung in eine HASH-Partitionierung umzuwandeln, dies muss durch das Anlegen einer neuen Datenbanktabelle und SELECT INSERT gelöst werden. Dabei darauf achten, zuerst die Daten zu kopieren, damit nichts verloren geht.

```
ALTER TABLE Bestellungen
ADD PARTITION (
    PARTITION p2025 VALUES LESS THAN (2026)
);
```

```
ALTER TABLE Bestellungen
REORGANIZE PARTITION
p2023, p2024
INTO (
    PARTITION p2023_2024 VALUES LESS THAN (2025)
);
```

```
ALTER TABLE Bestellungen
REORGANIZE PARTITION pMax INTO (
    PARTITION p2025 VALUES LESS THAN (2026),
    PARTITION pRest VALUES LESS THAN MAXVALUE
);
```

```
ALTER TABLE Bestellungen
DROP PARTITION p2022;
```

# Aufgabe 02

# Aufgabe 02

- 1. Erstellen Sie Partitionen basierend auf der Postleitzahl der Mitarbeitenden.**
- 2. Erstellen Sie Partitionen basierend auf dem Geburtsdatum der Besucher/-innen.**

Beachten Sie dabei die Einschränkungen in MySQL und legen Sie neue Datenbanktabellen ohne Primärschlüssel und ohne UNIQUE Index an. Die Datenbank erzeugt in diesem Fall eine versteckte Spalte „ROW ID“ welche den Primärschlüssel darstellt.

# SQL – Materialisierte Sichten

# SQL – Materialisierte Sichten

## Ansatz 1 – eine Relation (nicht normalisiert):

| <u>Kundennummer</u> | Vorname | Name       | Postleitzahl | Stadt    |
|---------------------|---------|------------|--------------|----------|
| 1                   | Max     | Mustermann | 68161        | Mannheim |
| 2                   | Marie   | Musterfrau | 68161        | Mannheim |

Nachteile Ansatz 1 zu Ansatz 2



Nachteile Ansatz 1 zu Ansatz 2

## Ansatz 2 – zwei Relationen (normalisiert):

| <u>Kundennummer</u> | Vorname | Name       | Postleitzahl |
|---------------------|---------|------------|--------------|
| 1                   | Max     | Mustermann | 68161        |
| 2                   | Marie   | Musterfrau | 68161        |

| <u>Postleitzahl</u> | Stadt    |
|---------------------|----------|
| 68161               | Mannheim |

# SQL – Materialisierte Sichten

Eine materialisierte Sicht ist ein View, welcher das Ergebnis allerdings physisch speichert. Im Gegensatz zu normalen Views, muss also nicht jedes Mal das Ergebnis neu berechnet werden, sondern wird einmalig berechnet und gespeichert.

## Vorteil

- Schnelle Zugriffe, da Ergebnis bereits berechnet

## Nachteil

- Basisrelation und materialisierte Sicht existieren parallel (physische Speicherung)  
→ Redundanz und Gefahr Inkonsistenz



sehr gut geeignet, für Relationen, deren Daten sich selten ändern

# SQL – Materialisierung

**CREATE MATERIALIZED VIEW** <View\_Name>  
[ **REFRESH** [ **FAST** | **COMPLETE** ] ]  
[ **ON COMMIT** | **ON DEMAND** ]  
**AS SELECT** ...

**REFRESH FAST** benötigt  
zusätzlich einen Änderungslog.

**CREATE MATERIALIZED VIEW LOG** ...

**Aktualisierung einer materialisierten View:**

**REFRESH MATERIALIZED VIEW** <View\_Name> ;

**Löschen einer materialisierten View:**

**DROP MATERIALIZED VIEW** <View\_Name>;

| Befehl           | Erklärung                                                 |
|------------------|-----------------------------------------------------------|
| REFRESH FAST     | nur Änderungen übernehmen                                 |
| REFRESH COMPLETE | Sicht neu berechnen                                       |
| ON COMMIT        | Automatische Aktualisierung bei Transaktionsende (COMMIT) |
| ON DEMAND        | Aktualisierung nur bei explizitem Aufruf                  |



MySQL unterstützt  
**CREATE  
MATERIALIZED  
VIEW** nicht.

# SQL – Materialisierung – MySQL

In MySQL gibt es keine materialisierten Views, daher muss dies mit Datenbanktabellen umgesetzt werden. Dazu wird eine neue Datenbanktabelle basierend auf dem SELECT der View erstellt.

**CREATE TABLE** <Datenbanktabelle\_Neu>  
**AS SELECT** ...

... → SQL-Query (i.d.R. für eine View)

## Aktualisierung der Daten:

**TRUNCATE TABLE** <Datenbanktabelle\_Neu>;  
→ Zunächst Daten aus der Datenbanktabelle löschen

**INSERT INTO** <Datenbanktabelle\_Neu>  
**SELECT** ...

... → SQL-Query (i.d.R. für eine View)

## Mögliche Lösung für On-Demand Aktualisierung:

- Trigger auf Datenbanktabelle (Basisrelationen der neuen Datenbanktabelle), um bei INSERT/UPDATE/DELETE die Änderungen zu übernehmen
- Achtung: Performance kann beeinträchtigt werden

# SQL – Materialisierung – Anwendungsbeispiel

Es gibt die Datenbanktabellen „Studierende“, „Belegung“ und „Kurse“. Möchte man nun wissen welcher Studierende welchen Kurs belegt hat (Klarnamen und Beschreibungen der Kurse, IDs sind in „Belegung“ bereits vorhanden) muss man alle drei Tabellen durch JOIN verknüpfen. Benötigt man diese Informationen mehrfach muss man diese Query mehrfach ausführen. Dies benötigt jedes Mal Zeit, dabei sind diese Informationen von hoher Wichtigkeit und müssen schnell abrufbar sein. Daher wurde entschieden, eine materialisierte View zu erzeugen.

## Schritt 1 – Erstellen der View, welche die drei Tabellen per JOIN kombiniert

```
CREATE TABLE Kursbelegung AS  
SELECT Vorname, Nachname, Kursbeschreibung, Kursnummer,  
Matrikelnummer  
FROM Studierende AS S  
INNER JOIN Belegung AS B ON B.Matrikelnummer = S.Matrikelnummer  
INNER JOIN Kurse AS K ON B.Kursnummer = K.Kursnummer;
```

## Schritt 2 – Selektieren der Daten aus der neuen Datenbanktabelle

```
SELECT * FROM Kursbelegung;
```

# Aufgabe 03

## Aufgabe 03

**Erstellen Sie zu den Views aus Datenbanken I – Aufgabe 11 materialisierte Sichten in Form von neuen Datenbanktabellen.**

# Fragen?

Mail: [andreas.nicola@sap.com](mailto:andreas.nicola@sap.com)